Statistics with R Programming

ASSIGNMENT 1

.

CO1

CO1: Design and construct appropriate R-based data representations using variables, data types, vectors, advanced data structures, matrix/array operations and R class mechanisms for data-oriented problems

---

Assignment 1 Title : Design and Develop a Scalable R Data Representation Framework for a Smart Retail Sales and Inventory Analytics System

---

Scenario — Smart Retail Sales and Inventory Analytics Platform
A national retail chain is developing an R-based Sales and Inventory Analytics Platform to integrate store registration details, product catalogue information, daily footfall, unit prices, stock levels, promotion flags, monthly sales records, sales channels and multi-dimensional sales data. The system must support approximately 500 stores and 40 product categories initially, but the architecture is expected to remain usable when the chain grows to 10,000 stores.

The analytics team has not fixed the R data representation. Store and product information may be represented using atomic vectors, character vectors, logical vectors, lists, factors, matrices, arrays, data frames and user-defined R classes. The team is concerned that an inappropriate representation may cause unintended coercion, inconsistent data types, difficult updates, weak validation, inefficient operations or poor maintainability.

You have been appointed as the R Data Architecture Team. Your responsibility is not merely to demonstrate R syntax. You must investigate the requirements, design a complete representation framework, construct reusable R components, integrate different R objects, validate the design experimentally and defend the final architecture using evidence.

The final solution must clearly distinguish between what R permits, what R automatically converts or coerces, and what is appropriate for the retail application. Students are expected to make design decisions independently and justify why their constructed solution is suitable for the given scenario.

Design Challenge

Create and validate an integrated R data-representation framework for the retail system. The framework must define how each category of information is stored, how multi-dimensional sales data is represented, how objects are validated and manipulated, and how S3, S4 or Reference Class mechanisms can be incorporated where a structured object model is required.

Your final design must be executable in R and must demonstrate interaction among the selected data structures rather than treating each R object as an isolated example. Every major design decision must be supported by experimental evidence and a clear explanation of the trade-offs involved.

TASK 1 – Formulate the Business Enquiry and Create the Data Architecture Blueprint 4 Marks

- Convert the scenario into a formal set of data-representation requirements. Identify the information categories (stores, products, prices, stock, sales and channels), relationships among them, expected operations and constraints.
- Create a data-architecture blueprint showing how variables, atomic/character/logical vectors, lists, factors, matrices, arrays and data frames will cooperate in the proposed system, and prepare a reproducible R session for it (working directory, R version, random seed and required packages).
- Formulate at least four enquiry questions that your design must answer experimentally, such as how representation affects coercion, indexing, validation, updating and analytical operations. Use basic arithmetic and built-in functions in R to estimate the number of cells and approximate storage required for 500 and 10,000 stores.
- Define at least five design criteria and use them as acceptance criteria for the final framework.

Evidence Expected

A requirements-to-representation blueprint, reproducible session set-up, enquiry questions, design criteria and a clear rationale showing how the proposed architecture addresses the scenario.

TASK 2 – Design a Robust Store and Product Core-Data Representation **4 Marks**

- Design and construct the R representation for store ID, store name, region, daily footfall, unit price, open/promotion status and other essential attributes of stores and products.
- Create suitable atomic, character and logical vectors and establish naming and type-consistency rules, including the handling of integer and double values.
- Construct validation logic that detects unsuitable values (for example negative prices or non-numeric footfall), missing values and unintended type conversion.

• Demonstrate at least two alternative representations for one important attribute, such as store region or promotion status, then create the final representation based on the identified requirements.

Evidence Expected
Executable R objects, validation rules, test cases, rejected alternatives and a justified final store-and-product data representation.

TASK 3 – Create a Coercion-Safe Sales Data Preparation Module **4 Marks**

• Create an R module that deliberately tests mixed numeric, integer, character and logical data from sales feeds and records the resulting types.
• Design rules or helper functions to prevent or detect harmful automatic coercion, and use logical-vector operations (such as &, |, !, any(), all(), which() and ifelse()) to flag invalid or suspicious records before the data enter the analytics system.
• Construct test cases containing valid, invalid, missing and mixed-type values and demonstrate how the module responds.
• Explain how your design preserves semantic meaning while still using R's native data structures.

Evidence Expected
A reusable coercion-testing/validation component, logical-vector flagging rules, experimental outputs, test cases, and evidence that the constructed rules prevent or expose unintended conversion.

TASK 4 – Construct a Store-by-Product Sales Matrix Processing Engine **4 Marks**

• Create a matrix-based component representing unit sales (or revenue) of stores across product categories.
• Develop R operations for indexing, row/column summaries, transposition and at least one meaningful matrix operation relevant to retail analytics, such as multiplying a units-sold matrix by a unit-price vector to obtain store revenue.
• Construct a mechanism for connecting matrix positions with store and product identifiers (row and column names) without losing interpretability.
• Create and justify a conversion strategy for moving between the matrix representation and a more metadata-rich representation such as a data frame.

Evidence Expected
Working matrix component, meaningful operations, mapping/identifier strategy, conversion code and evidence demonstrating correct results.

TASK 5 – Create a Multi-Dimensional Sales Array Model 4 Marks

- Design and construct a three-dimensional or higher-dimensional array for stores, product categories and months or sales channels (for example in-store, online and mobile app).
- Develop indexing and calculation operations, for example using apply(), sum(), mean() and aperm(), that retrieve and summarize selected dimensions.
- Create a method for assigning meaningful dimension names and demonstrate why the dimensions remain interpretable.
- Construct at least one transformation from the array into another R representation suitable for reporting or analysis.

Evidence Expected

Executable array model, named dimensions, retrieval/calculation operations, transformation procedure and evidence of correct multi-dimensional handling.

TASK 6 – Build an Integrated List, Factor and Data-Frame Layer 5 Marks

- Create a heterogeneous store record using a list (including nested elements) and design factors for categorical variables such as region, store format, product category or stock-risk level.
- Construct a data frame that integrates appropriate vectors, factors and numerical variables into an analysis-ready table.
- Develop operations for adding, updating, selecting and summarising store, product or sales records.
- Create rules for deciding when a variable should remain a character vector and when it should be represented as a factor, including the use of ordered factors and level management.
- Demonstrate that the constructed layer can exchange data with the matrix and array components developed earlier.

Evidence Expected

An integrated R data layer, reusable operations, categorical-data design rules, and evidence showing successful interaction among lists, factors, data frames, matrices and arrays.

TASK 7 – Design an Object-Oriented Store/Product Model Using R Classes 5 Marks

- Create a realistic Store, Product or SalesRecord object model using S3, S4 or Reference Class mechanisms.
- Implement the selected class structure with appropriate attributes (slots or fields) and behaviour for the scenario.
- Create at least two operations/methods or functions that act on the constructed object, such as updating stock, calculating revenue, flagging reorder status, generating a summary or validating a sales record.
- Construct a small comparative prototype using a second class mechanism and use the investigation to justify the final class design.

• Explain how the chosen class mechanism improves structure, validation, reuse or maintainability for the proposed system.

Evidence Expected
Constructed class-based prototype(s), working methods/functions, test cases, comparison evidence and a technically justified final class architecture.

TASK 8 – Develop a Reusable R Function Library for the Framework 4 Marks

• Create a minimum of four reusable R functions supporting the proposed framework, including at least one function for validation and one for sales calculation or summary.
• Use default arguments where appropriate and explicitly design the return object for each function.
• At least one function must return a complex object such as a list or data frame and must integrate with another component of the framework.
• Create test cases that demonstrate valid, boundary and invalid inputs and document the expected behaviour.

Evidence Expected
A reusable function library, documented arguments/return values, test cases and evidence of integration with the constructed data framework.

TASK 9 – Integrate, Test and Refine the Complete R Framework 4 Marks

• Integrate the components created in Tasks 2–8 into one coherent retail analytics prototype.
• Generate or construct a representative dataset and execute a realistic workflow from raw store and sales information to validated, structured and analysis-ready objects.
• Design systematic test cases covering correctness, missing values, invalid categories, dimension errors and type/coercion problems.
• Record failures encountered during development and show how the design was refined to address them.

Evidence Expected
A reproducible integrated prototype, workflow demonstration, systematic test cases, failure/refinement evidence and final corrected implementation

TASK 10 – Create an Evidence-Based Scalability and Maintainability Evaluation 4 Marks

- Construct datasets representing at least 500 and 10,000 stores, with an appropriate number of product categories and sales dimensions.

- Experiment with at least three important operations from the proposed framework and record meaningful measures such as execution time, object dimensions, memory-related observations where feasible, or number of processing steps.
- Create a comparison table and at least one suitable graph or structured visualization.
- Use the evidence to refine the architecture where necessary rather than simply reporting performance results.

Evidence Expected

Reproducible experimental setup, comparison table, visualization, interpretation and evidence of at least one design refinement based on the experiments.

TASK 11 – Defend the Final Architecture Under a Changed Requirement 4 Marks

- Changed Requirement: The retail chain now plans to integrate 10,000 stores, 100 product categories, multiple sales channels (in-store, online and mobile app) and frequent store-level stock and price updates. The analytics team also requires the framework to remain easy to validate, transform and extend.
- Create a revised version of your architecture by deciding which components should be retained, modified, replaced or redesigned.
- Construct at least one new or modified component that directly addresses the changed requirement and demonstrate its integration with the existing framework.
- Defend the final architecture using evidence from your experiments, object behaviour, scalability observations, validation requirements, maintainability and semantic correctness.
- Identify at least two trade-offs or limitations that remain in your final design and propose future improvements.

Evidence Expected

A revised executable architecture, demonstrated adaptation, evidence-based defence, explicit trade-offs and a clear explanation of how the final design satisfies the changed requirements.

TASK 12 – Document, Package and Present the Final Framework 4 Marks

- Prepare a data dictionary for the final framework listing every major R object with its name, class, type, dimensions or length, purpose and validation rule (using functions such as class(), typeof(), dim(), length() and str()).
- Construct a single reproducible R script, or an ordered set of scripts, that runs from a fresh R session to rebuild all objects, test cases and outputs, using set.seed(), commented sections and recorded session information.
- Create a summary-report function that returns a structured list or data frame consolidating the key results of the framework, such as store totals, product summaries, validation outcomes and test-case status.
- Write a short evidence-based recommendation for retail management explaining which R representation (vector, list, factor, matrix, array, data frame or class) should be used for which kind of information, and reflect on at least two lessons learned during development.

Evidence Expected

A data dictionary, reproducible run script with session information, summary-report output, management recommendation and a reflective note linking design decisions to the evidence produced in Tasks 1–11.

MARK DISTRIBUTION

Task	Marks
TASK 1 – Formulate	4
TASK 2 – Design	4
TASK 3 – Create	4
TASK 4 – Construct	4
TASK 5 – Create	4
TASK 6 – Build	5
TASK 7 – Design	5
TASK 8 – Develop	4
TASK 9 – Integrate,	4
TASK 10 – Create	4
TASK 11 – Defend	4
TASK 12 – Document,	4
TOTAL	50

STATISTICS WITH R PROGRAMMING

ASSIGNMENT 1

Design and Develop a Scalable R Data Representation Framework for a Smart Retail Sales and Inventory Analytics System

Course Outcome: CO1

Total Marks: 50

Name: P.Veeraswami

Register Number: 192311253

Department / Class: CSE

Date:

CONTENTS

1. Task 1 – Business enquiry and architecture blueprint

2. Task 2 – Store and product core-data representation

3. Task 3 – Coercion-safe sales data preparation

4. Task 4 – Store-by-product sales matrix

5. Task 5 – Multi-dimensional sales array

6. Task 6 – Integrated list, factor and data-frame layer

7. Task 7 – Object-oriented Store/Product model

8. Task 8 – Reusable R function library

9. Task 9 – Integration, testing and refinement

10. Task 10 – Scalability and maintainability evaluation

11. Task 11 – Architecture under changed requirement

12. Task 12 – Documentation, packaging and presentation

13. Consolidated executable R script

14. Conclusion

INTRODUCTION

The assignment develops an R-based data-representation framework for a national retail chain. The system must store store-registration details, product catalogue data, footfall, prices, stock, promotions, monthly sales and sales-channel information. The initial scale is approximately 500 stores and 40 product categories, with a planned expansion to 10,000 stores.

The main design principle is to select a representation according to the meaning and operations of the data. Atomic vectors store homogeneous values; lists store heterogeneous records; factors represent controlled categories; data frames hold analysis-ready rectangular data; matrices support numerical store-by-product calculations; arrays represent three or more dimensions; and S3/S4 classes organize reusable behaviour.

The source assignment specifies the requirements and tasks but does not provide an actual retail dataset or measured execution timings. Therefore, all sample data and printed results in this answer are explicitly illustrative. The R code is designed to be reproducible and can be executed to obtain machine-specific timings and outputs.

TASK 1 – Formulate the Business Enquiry and Create the Data Architecture Blueprint (4 Marks)

Question / requirement: Convert the scenario into formal data-representation requirements; create a blueprint and reproducible R session; formulate at least four experimental enquiry questions; estimate cells and approximate storage for 500 and 10,000 stores; and define at least five acceptance criteria.

1.1 Business requirements

Information category	Required fields / meaning	Recommended R representation
Stores	store_id, store_name, region, format, daily_footfall, open_status	Character, integer/double and logical vectors combined in a data frame
Products	product_id, product_name, category, unit_price	Character and double vectors; category as factor where levels are controlled
Inventory	store_id, product_id, stock_qty, reorder_level	Integer vectors in a data frame; matrix for a dense store-by-product view
Sales	store, product, date/month, units, revenue	Data frame for transaction records; matrix for store-product summaries
Channels	in_store, online, mobile_app	Factor for transaction channel; array dimension for aggregated channel data
Multi-dimensional totals	store × product × month/channel	Three-dimensional array or higher-dimensional array

Relationships: each store has many inventory and sales records; each product can appear in many stores; each sales record refers to one valid store, one valid product and one sales channel. Store and product identifiers are keys and must not be replaced by row positions.

1.2 Architecture blueprint

Layer	R object	Responsibility
Raw input	Character/numeric/logical vectors	Preserve incoming fields and inspect types before conversion
Core entities	Data frames and nested lists	Store and product master data; heterogeneous individual records
Categories	Factors	Controlled region, format, category and channel levels
Analytical engine	Matrices	Fast numerical store × product summaries
Time/channel cube	Arrays	Store × product × month or channel calculations
Business rules	S3/S4 class and functions	Validation, stock updates, revenue and summary methods
Output	Data frames and report lists	Reporting, export and management summaries

Blueprint flow: Raw feeds → type inspection → validation → store/product master tables → sales data frame → matrix/array aggregation → class-based business methods → summary report.

1.3 Reproducible R session

```
# Run at the beginning of the project
setwd("C:/RProjects/SmartRetail")
set.seed(2026)
R.version.string
sessionInfo()

required_packages <- c("utils", "stats")
# Base R is sufficient for the core framework.
# Optional reshaping/plotting packages can be installed when required:
# install.packages(c("tidyr", "ggplot2"))
```

A fixed working directory makes input and output locations predictable. set.seed(2026) makes random test-data generation reproducible. Base R is deliberately preferred so the framework has few external dependencies.

1.4 Experimental enquiry questions

1. Does combining numeric, character and logical values in one atomic vector cause coercion, and how can the original meaning be preserved?
2. How do matrix indexing and row/column names affect the accuracy of store and product retrieval?
3. Which validation rules detect missing IDs, negative prices, negative stock and invalid categories before analysis?
4. How do matrix and array summaries change when the number of stores, products and channels increases?
5. Does a structured class reduce the risk of invalid stock updates compared with directly changing a numeric vector?

1.5 Cell and storage estimates

For a simple store-by-product matrix with 40 categories, the number of numeric cells is stores × 40. For the expanded model with 100 products and three sales channels, the illustrative array size is stores × 100 × 3. R numeric values are generally stored as 8-byte doubles; this estimate excludes object headers, names, dimensions, metadata and temporary copies.

Scenario	**Calculation**	**Cells**	**Approx. numeric payload**
500 stores × 40 products	500 × 40	20,000	20,000 × 8 = 160,000 bytes ≈ 0.153 MiB
10,000 stores × 40 products	10,000 × 40	400,000	3,200,000 bytes ≈ 3.05 MiB
10,000 × 100 products × 3 channels	10,000 × 100 × 3	3,000,000	24,000,000 bytes ≈ 22.89 MiB

These are payload estimates, not complete object sizes. Use object.size() in R for a more realistic measurement. Sparse transaction data may be much smaller as a data frame than a mostly empty dense matrix.

1.6 Acceptance criteria

Criterion	**Acceptance condition**
Type safety	IDs remain character; quantities are integer where appropriate; prices and revenue are numeric; flags are logical.
Completeness	Required IDs and key fields contain no missing values.
Validity	Prices, footfall and stock cannot be negative; category/channel values must belong to allowed levels.
Traceability	Every matrix/array row and column maps to explicit

	store/product/channel identifiers.
Reproducibility	The complete script runs from a fresh R session with a fixed seed.
Scalability	The architecture supports 10,000 stores and 100 products without changing the meaning of fields.
Maintainability	Validation and business operations are reusable functions or class methods.

TASK 2 – Design a Robust Store and Product Core-Data Representation (4 Marks)

Question / requirement: Construct store and product representations using suitable atomic, character and logical vectors; establish type rules; validate unsuitable and missing values; compare at least two alternatives for one attribute; and justify the final representation.

2.1 Core objects and type rules

```
stores <- data.frame(
  store_id = c("S001", "S002", "S003"),
  store_name = c("Central", "North Mall", "Airport"),
  region = c("South", "North", "South"),
  daily_footfall = c(1250L, 980L, 1500L),
  open_status = c(TRUE, TRUE, FALSE),
  stringsAsFactors = FALSE
)

products <- data.frame(
  product_id = c("P001", "P002", "P003"),
  product_name = c("Rice", "Tea", "Soap"),
  category = factor(c("Grocery", "Beverage", "Personal Care")),
  unit_price = c(65.50, 120.00, 32.00),
  stock_qty = c(250L, 80L, 0L),
  promotion_status = c(FALSE, TRUE, FALSE)
)

str(stores)
str(products)
```

Identifiers are stored as character strings because IDs are labels, not quantities. Footfall and stock counts are integers. Prices are doubles to preserve decimal values. Open and promotion flags are logical values.

2.2 Validation function

```
validate_core <- function(stores, products) {
  store_errors <- character()
```

```
  product_errors <- character()

  if (anyNA(stores$store_id) || any(stores$store_id == ""))
  store_errors <- c(store_errors, "Missing or blank store ID")
  if (anyDuplicated(stores$store_id))
    store_errors <- c(store_errors, "Duplicate store ID")
  if (!is.numeric(stores$daily_footfall) ||
      any(stores$daily_footfall < 0, na.rm = TRUE))
  store_errors <- c(store_errors, "Invalid daily footfall")
  if (!is.logical(stores$open_status))
    store_errors <- c(store_errors, "Open status must be logical")

  if (anyNA(products$product_id) || any(products$product_id == ""))
  product_errors <- c(product_errors, "Missing or blank product ID")
  if (anyDuplicated(products$product_id))
    product_errors <- c(product_errors, "Duplicate product ID")
  if (!is.numeric(products$unit_price) ||
      any(products$unit_price < 0, na.rm = TRUE))
  product_errors <- c(product_errors, "Invalid unit price")
  if (!is.numeric(products$stock_qty) ||
      any(products$stock_qty < 0, na.rm = TRUE))
    product_errors <- c(product_errors, "Invalid stock quantity")

  list(valid = length(store_errors) == 0 &&
               length(product_errors) == 0,
       store_errors = store_errors,
       product_errors = product_errors)
}

validate_core(stores, products)
```

The returned list separates the overall result from store-specific and product-specific errors. In production, validation should stop the pipeline or quarantine invalid rows instead of silently deleting them.

2.3 Alternative representations: region

Representation	Example	Assessment
Character vector	c('South','North','South')	Flexible for incoming or changing labels; permits spelling inconsistencies.
Factor	factor(c('South','North'), levels=c('North','South'))	Controls allowed categories and stores them as integer codes plus level labels.
Final decision	Factor in analysis table; character in raw ingestion	Raw data is preserved as received, then validated and converted to a controlled factor.

Promotion status should be logical rather than text such as 'Yes'/'No'. A logical vector enables direct filtering and avoids spelling variants. Missing status remains NA and must be handled explicitly.

TASK 3 – Create a Coercion-Safe Sales Data Preparation Module (4 Marks)

Question / requirement: Test mixed numeric, integer, character and logical feed values; create rules to prevent or detect harmful coercion; use logical-vector operations to flag suspicious records; and demonstrate valid, invalid, missing and mixed-type cases.

3.1 Demonstrating automatic coercion

```
mixed <- c(10L, 12.5, "15", TRUE)
mixed
typeof(mixed)
class(mixed)

safe_fields <- list(
  units = c(10L, 12L, 15L),
  unit_price = c(65.5, 120.0, 32.0),
  promotion = c(TRUE, FALSE, TRUE),
  product_id = c("P001", "P002", "P003")
)
lapply(safe_fields, typeof)
```

An atomic vector can contain only one underlying type. In a mixed vector, R promotes values to a common type; here the character value forces the vector to character. A list or data frame is appropriate when fields have different meanings and types.

3.2 Safe sales-feed validator

```
validate_sales_feed <- function(feed) {
  required <- c("store_id", "product_id", "units",
                "unit_price", "channel", "promotion")
  missing_columns <- setdiff(required, names(feed))
  if (length(missing_columns) > 0)
    stop("Missing columns: ", paste(missing_columns, collapse = ", "))

  # Convert only after checking that text can be parsed as numeric.
  units_text <- as.character(feed$units)
  price_text <- as.character(feed$unit_price)
  units_num <- suppressWarnings(as.numeric(units_text))
  price_num <- suppressWarnings(as.numeric(price_text))
```

```
  bad_units <- is.na(units_num) | units_num < 0 |
            units_num != floor(units_num)
  bad_price <- is.na(price_num) | price_num < 0
  bad_channel <- is.na(feed$channel) |
              !feed$channel %in% c("in_store", "online", "mobile_app")
  bad_promotion <- is.na(feed$promotion) |
                !as.character(feed$promotion) %in% c("TRUE", "FALSE")

  suspicious <- bad_units | bad_price | bad_channel | bad_promotion |
              is.na(feed$store_id) | is.na(feed$product_id)

  result <- data.frame(feed, units_checked = units_num,
                       price_checked = price_num,
                       invalid = suspicious)
  result
}

sales_raw <- data.frame(
  store_id = c("S001", "S001", "S002", NA),
  product_id = c("P001", "P002", "P003", "P001"),
  units = c("4", "-2", "many", "3"),
  unit_price = c("65.5", "120", "32", "65.5"),
  channel = c("in_store", "online", "web", "mobile_app"),
  promotion = c("FALSE", "TRUE", "FALSE", "TRUE"),
  stringsAsFactors = FALSE
)
checked_sales <- validate_sales_feed(sales_raw)
checked_sales
```

The validator uses | to combine invalid conditions, ! to negate allowed values, %in% for category membership and is.na() for missing data. any() and all() can summarize flags; which() identifies the row positions requiring review.

```
any(checked_sales$invalid)
all(!checked_sales$invalid)
which(checked_sales$invalid)
```

Expected illustrative interpretation: rows containing negative units, non-numeric units, an unknown channel or a missing store ID are flagged. The original feed columns are retained so the analyst can trace each rejection.

TASK 4 – Construct a Store-by-Product Sales Matrix Processing Engine (4 Marks)

Question / requirement: Create a matrix for units or revenue; demonstrate indexing, row/column summaries, transpose and a meaningful matrix operation; map positions to identifiers; and convert to a metadata-rich data frame.

4.1 Matrix construction and operations

```
units_matrix <- matrix(
  c(12, 8, 5,
    7, 10, 4,
    15, 6, 9),
  nrow = 3, byrow = TRUE,
  dimnames = list(
    store_id = c("S001", "S002", "S003"),
    product_id = c("P001", "P002", "P003")
  )
)

units_matrix
units_matrix["S001", "P002"]       # One store-product cell
rowSums(units_matrix)              # Total units by store
colSums(units_matrix)              # Total units by product
t(units_matrix)                    # Transposed view

price_vector <- c(P001 = 65.5, P002 = 120, P003 = 32)
revenue_by_store <- units_matrix %*% price_vector
revenue_by_store
```

The matrix uses store IDs as row names and product IDs as column names. The named price vector must have the same product names and order as the matrix columns. Matrix multiplication calculates each store's total revenue from units multiplied by unit prices.

4.2 Convert matrix to long data frame

```
sales_long <- as.data.frame(as.table(units_matrix))
names(sales_long) <- c("store_id", "product_id", "units_sold")
sales_long$store_id <- as.character(sales_long$store_id)
sales_long$product_id <- as.character(sales_long$product_id)
sales_long$revenue <- sales_long$units_sold *
unname(price_vector[sales_long$product_id])
sales_long
```

A matrix is compact and efficient for dense numeric calculations, but it cannot naturally hold text metadata in each cell. A data frame is preferred for reporting because it can combine IDs, dates, categories, quantities and prices. Conversion should retain explicit identifiers.

TASK 5 – Create a Multi-Dimensional Sales Array Model (4 Marks)

Question / requirement: Construct a three-dimensional or higher-dimensional array for stores, product categories and months or channels; use indexing, apply(), sum(), mean() and aperm(); assign meaningful dimension names; and transform the array for reporting.

5.1 Three-dimensional array

```
sales_array <- array(
  data = c(12, 8, 5, 7, 10, 4,
           15, 6, 9, 11, 7, 3,
           9, 5, 6, 8, 12, 4),
  dim = c(3, 3, 2),
  dimnames = list(
    store = c("S001", "S002", "S003"),
    product = c("P001", "P002", "P003"),
    month = c("Jan", "Feb")
  )
)

sales_array["S001", , "Jan"]
apply(sales_array, 1, sum)      # Total units by store
apply(sales_array, 2, sum)      # Total units by product
apply(sales_array, 3, sum)      # Total units by month
mean(sales_array)
aperm(sales_array, c(3, 1, 2))
```

The dimensions are store × product × month. Named dimensions allow an analyst to retrieve a particular store, product or month without relying on undocumented numeric positions. apply() aggregates over the dimensions not retained.

5.2 Array to reporting table

```
array_long <- as.data.frame(as.table(sales_array))
names(array_long) <- c("store_id", "product_id", "month", "units_sold")
array_long$revenue <- array_long$units_sold *
unname(price_vector[as.character(array_long$product_id)])
head(array_long)
```

The long-form data frame is suitable for filtering, joining to store/product metadata, plotting and exporting. The array remains useful for compact numerical calculations when the dimensions are regular.

TASK 6 – Build an Integrated List, Factor and Data-Frame Layer (5 Marks)

Question / requirement: Create a heterogeneous nested store list; construct factors; integrate vectors, factors and numeric variables into a data frame; support add/update/select/summarise operations; define character-versus-factor rules; and exchange data with matrix/array components.

6.1 Nested store record

```
store_record <- list(
  identity = list(store_id = "S001", store_name = "Central"),
  location = list(region = "South", city = "Chennai"),
  operations = list(open_status = TRUE, daily_footfall = 1250L),
  inventory = data.frame(
    product_id = c("P001", "P002"),
    stock_qty = c(250L, 80L)
  )
)
str(store_record)
```

A list is suitable because a store record contains nested information with different types and structures. It is not restricted to a single atomic type.

6.2 Factors and analysis-ready data frame

```
store_analysis <- data.frame(
  store_id = stores$store_id,
  region = factor(stores$region,
                  levels = c("North", "South", "East", "West")),
  store_format = factor(c("Mall", "High Street", "Airport")),
  daily_footfall = stores$daily_footfall,
  open_status = stores$open_status
)

stock_risk <- factor(
  c("Low", "Medium", "High"),
  levels = c("Low", "Medium", "High"),
  ordered = TRUE
)
store_analysis$stock_risk <- stock_risk
```

Use a character vector for free-form labels, identifiers, names and raw incoming values. Use a factor for a finite, controlled set of categories such as region, store format and sales channel. Use an ordered factor only when the categories have a meaningful order, such as Low < Medium < High.

6.3 Add, update, select and summarise

```
# Add a new store
new_store <- data.frame(
store_id = "S004",
  region = factor("West", levels = levels(store_analysis$region)),
  store_format = factor("Mall", levels = levels(store_analysis$store_format)),
  daily_footfall = 700L,
  open_status = TRUE,
  stock_risk = ordered("Medium", levels = levels(stock_risk))
)
store_analysis <- rbind(store_analysis, new_store)

# Update a value
store_analysis$daily_footfall[store_analysis$store_id == "S004"] <- 750L

# Select open stores and summarise footfall
open_stores <- subset(store_analysis, open_status)
aggregate(daily_footfall ~ region, data = store_analysis, FUN = mean)
```

When adding a new category to a factor, first update its levels or the new value may become NA. For example, levels(store_analysis$region) <- c(levels(store_analysis$region), 'Central') before assigning a new Central category.

6.4 Exchange with matrix and array

```
# Matrix values become a long data frame and can be joined to metadata.
matrix_table <- as.data.frame(as.table(units_matrix))
names(matrix_table) <- c("store_id", "product_id", "units_sold")
matrix_table$store_id <- as.character(matrix_table$store_id)
matrix_table$product_id <- as.character(matrix_table$product_id)

# Array values become a reporting table.
array_table <- as.data.frame(as.table(sales_array))
names(array_table) <- c("store_id", "product_id", "month", "units_sold")
```

The list is the flexible entity-record layer, factors enforce controlled categories, data frames are the integration layer, and matrix/array objects supply numerical aggregation. These structures cooperate through stable IDs and explicit conversion.

TASK 7 – Design an Object-Oriented Store/Product Model Using R Classes (5 Marks)

Question / requirement: Create a Store, Product or SalesRecord object using S3, S4 or Reference Classes; implement at least two methods; compare a second class mechanism; and justify the final design.

7.1 S3 Store class

```
new_store <- function(id, name, stock, reorder_level = 10L) {
  stopifnot(is.character(id), length(id) == 1, !is.na(id))
  stopifnot(is.numeric(stock), length(stock) == 1,
            stock >= 0, stock == floor(stock))
  structure(
    list(id = id, name = name, stock = as.integer(stock),
         reorder_level = as.integer(reorder_level)),
    class = "RetailStore"
  )
}

print.RetailStore <- function(x, ...) {
  cat("Store:", x$id, "-", x$name, "\n")
  cat("Stock:", x$stock, " Reorder level:", x$reorder_level, "\n")
}

update_stock <- function(x, quantity) {
  stopifnot(inherits(x, "RetailStore"))
  stopifnot(is.numeric(quantity), length(quantity) == 1,
            quantity == floor(quantity))
  if (x$stock + quantity < 0) stop("Stock cannot become negative")
  x$stock <- as.integer(x$stock + quantity)
  x
}

needs_reorder <- function(x) {
  stopifnot(inherits(x, "RetailStore"))
  x$stock <= x$reorder_level
}

central <- new_store("S001", "Central", 8L, 10L)
central
central <- update_stock(central, 20L)
needs_reorder(central)
```

S3 is lightweight and flexible. A class is assigned through the class attribute, and generic functions such as print() dispatch to methods based on the object's class. The constructor performs basic validation, while update_stock() prevents a negative stock balance.

7.2 Comparative S4 prototype

```
setClass(
  "RetailProduct",
  slots = c(
    id = "character",
    name = "character",
    price = "numeric",
    stock = "integer"
  ),
  validity = function(object) {
    if (length(object@id) != 1 || object@id == "")
    return("Product ID must be one non-empty value")
    if (length(object@price) != 1 || object@price < 0)
    return("Price must be one non-negative value")
    if (length(object@stock) != 1 || object@stock < 0L)
    return("Stock must be one non-negative integer")
    TRUE
  }
)

tea <- new("RetailProduct", id = "P002", name = "Tea",
           price = 120, stock = 80L)
validObject(tea)
```

Feature	**S3**	**S4**
Structure	Flexible list with class attribute	Formal class with declared slots
Validation	Usually implemented in constructor/functions	Supports validity checks and declared slot types
Complexity	Simple and easy to extend	More formal and strict
Best fit here	Store operations and lightweight business methods	Product records needing strict slot/type rules

Final choice: use S3 for the main RetailStore behaviour because the prototype needs simple, reusable methods and flexible integration with data frames. Use S4 where strict formal structure is especially valuable. Reference Classes are mutable, but shared mutable state can make updates harder to trace; the proposed framework prefers explicit returned objects.

TASK 8 – Develop a Reusable R Function Library for the Framework (4 Marks)

Question / requirement: Create at least four reusable functions, including validation and sales calculation/summary; use defaults where appropriate; define return objects; and test valid, boundary and invalid inputs.

8.1 Reusable functions

```
# 1. Validate price and quantity vectors
validate_items <- function(price, quantity, allow_zero = TRUE) {
  if (!is.numeric(price) || !is.numeric(quantity))
    return(list(valid = FALSE, message = "Inputs must be numeric"))
  if (length(price) != length(quantity))
    return(list(valid = FALSE, message = "Lengths must match"))
  lower_fail <- if (allow_zero) price < 0 | quantity < 0
                else price <= 0 | quantity < 0
  bad <- is.na(price) | is.na(quantity) | lower_fail
  list(valid = !any(bad), invalid_rows = which(bad),
       message = if (any(bad)) "Invalid item values" else "Valid")
}

# 2. Calculate line revenue
calculate_revenue <- function(quantity, price, discount = 0) {
  stopifnot(length(quantity) == length(price))
  stopifnot(all(quantity >= 0, na.rm = TRUE),
            all(price >= 0, na.rm = TRUE),
            discount >= 0, discount <= 1)
  data.frame(quantity = quantity, price = price,
             gross = quantity * price,
             net = quantity * price * (1 - discount))
}

# 3. Summarise store sales
summarise_store_sales <- function(sales, store_col = "store_id") {
  if (!all(c(store_col, "units_sold", "revenue") %in% names(sales)))
  stop("Required sales columns are missing")
  aggregate(cbind(units_sold, revenue),
            sales[store_col], sum, na.rm = TRUE)
}

# 4. Flag reorder items
flag_reorder <- function(stock, reorder_level = 10L) {
  if (!is.numeric(stock) || any(stock < 0, na.rm = TRUE))
  stop("Stock must be non-negative numeric values")
  data.frame(stock = stock, reorder_level = reorder_level,
             reorder = stock <= reorder_level)
}
```

```
# 5. Build a compact report list
build_retail_report <- function(stores, products, sales) {
  list(
    store_count = nrow(stores),
    product_count = nrow(products),
    sales_rows = nrow(sales),
    total_units = sum(sales$units_sold, na.rm = TRUE),
    total_revenue = sum(sales$revenue, na.rm = TRUE),
    revenue_by_store = summarise_store_sales(sales)
  )
}
```

8.2 Test cases and expected behaviour

```
validate_items(c(10, 20), c(1, 2))       # Valid
validate_items(c(0, 20), c(0, 2))        # Valid boundary by default
validate_items(c(-5, 20), c(1, 2))       # Invalid negative price
calculate_revenue(c(2, 3), c(50, 20), discount = 0.10)
flag_reorder(c(2, 10, 25), reorder_level = 10)
```

The validation function returns a list with a logical result and invalid row positions. The revenue function returns a data frame, making it easy to append or summarize. The report function returns a nested list that integrates several framework components.

TASK 9 – Integrate, Test and Refine the Complete R Framework (4 Marks)

Question / requirement: Integrate Tasks 2–8 into a coherent prototype; construct representative data; run a workflow from raw data to analysis-ready objects; test correctness, missing values, invalid categories, dimension errors and coercion; and record refinements.

9.1 Integrated workflow

```
# Representative input transaction feed
sales_feed <- data.frame(
  store_id = c("S001", "S001", "S002", "S003"),
  product_id = c("P001", "P002", "P001", "P003"),
  units = c("4", "2", "5", "3"),
  unit_price = c("65.5", "120", "65.5", "32"),
  channel = c("in_store", "online", "mobile_app", "in_store"),
  promotion = c("FALSE", "TRUE", "FALSE", "FALSE"),
  stringsAsFactors = FALSE
)

validated <- validate_sales_feed(sales_feed)
valid_rows <- validated[!validated$invalid, ]
```

```
sales_ready <- data.frame(
  store_id = valid_rows$store_id,
  product_id = valid_rows$product_id,
  units_sold = as.integer(valid_rows$units_checked),
  unit_price = valid_rows$price_checked,
  channel = factor(valid_rows$channel,
                   levels = c("in_store", "online", "mobile_app")),
  promotion = as.logical(valid_rows$promotion)
)
sales_ready$revenue <- sales_ready$units_sold * sales_ready$unit_price

# Aggregate by store and product
sales_matrix <- xtabs(units_sold ~ store_id + product_id,
                      data = sales_ready)
sales_report <- summarise_store_sales(
  transform(sales_ready, revenue = revenue), "store_id"
)
sales_report
```

9.2 Systematic test plan

Test	Input / action	Expected result
Correct record	Valid IDs, non-negative numeric units/price and allowed channel	Accepted; numeric columns remain numeric
Missing key	store_id = NA	Flagged and excluded from analysis until corrected
Negative value	units = -2 or price = -10	Flagged as invalid
Unknown category	channel = 'web'	Flagged because it is not an allowed channel
Mixed type	units = c('2','many')	'many' becomes NA during safe numeric parsing and is flagged
Dimension error	Price vector length does not equal matrix columns	Matrix multiplication fails; check names and lengths before calculation

9.3 Failure and refinement record

Observed risk / failure	Refinement
Mixed atomic vectors coerce numeric/logical values to character	Keep fields in separate data-frame columns or a list; parse each field explicitly
Unknown factor values become NA	Validate against allowed levels before factor conversion and manage levels deliberately
Matrix values lose business metadata	Keep stable row/column identifiers and convert to a long data frame for reporting
Array indexing becomes unclear	Assign dimension names and use named indexing
Invalid stock update could create negative inventory	Route updates through validated class methods

The integrated prototype should be executed from top to bottom. The raw feed remains available for audit, while only validated records enter the analysis-ready table. A production system should additionally log rejected rows, timestamps and correction status.

TASK 10 – Create an Evidence-Based Scalability and Maintainability Evaluation (4 Marks)

Question / requirement: Construct datasets for at least 500 and 10,000 stores; experiment with at least three operations; record timing, dimensions and memory observations; provide a comparison table and graph; and refine the architecture using evidence.

10.1 Reproducible benchmark code

```
make_sales_matrix <- function(n_stores, n_products = 40L) {
  matrix(
    sample(0:20, n_stores * n_products, replace = TRUE),
    nrow = n_stores,
    dimnames = list(
      paste0("S", seq_len(n_stores)),
      paste0("P", seq_len(n_products))
    )
  )
}

benchmark_one <- function(n_stores, n_products = 40L) {
  x <- make_sales_matrix(n_stores, n_products)
  price <- setNames(runif(n_products, 10, 200), colnames(x))

  t1 <- system.time(rowSums(x))[["elapsed"]]
  t2 <- system.time(colSums(x))[["elapsed"]]
  t3 <- system.time(x %*% price)[["elapsed"]]

  data.frame(
    stores = n_stores, products = n_products,
    cells = length(x),
    size_MiB = as.numeric(object.size(x)) / 1024^2,
    row_summary_seconds = t1,
    column_summary_seconds = t2,
    revenue_seconds = t3
  )
}

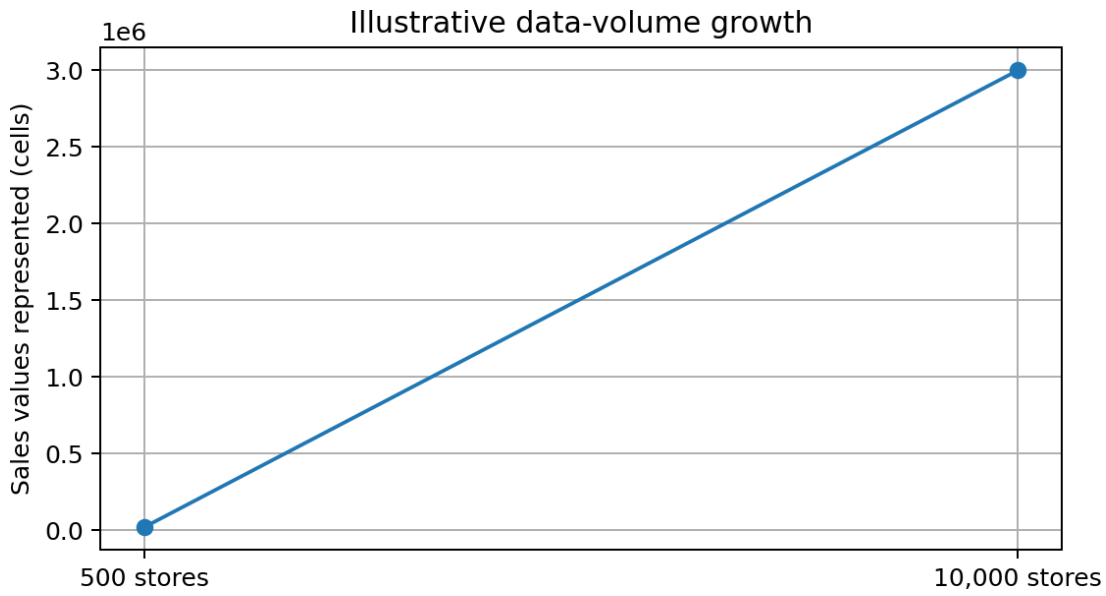
set.seed(2026)
benchmark_results <- rbind(
benchmark_one(500, 40),
  benchmark_one(10000, 40),
  benchmark_one(10000, 100)
)
```

benchmark_results

system.time() records elapsed execution time on the machine running the script. object.size() estimates the R object's memory footprint. The values should be copied from the actual R execution into the report; they are machine- and workload-dependent, so this document does not invent benchmark timings.

10.2 Structural comparison

Configuration	Stores	Products	Cells	Approx. double payload
Initial	500	40	20,000	0.153 MiB
Growth	10,000	40	400,000	3.05 MiB
Changed requirement	10,000	100 × 3 channels	3,000,000	22.89 MiB



Interpretation: the numeric payload grows linearly with each dimension. Moving from 500 × 40 to 10,000 × 100 × 3 increases the cell count from 20,000 to 3,000,000, a 150-fold increase. This remains manageable for a compact numeric array in many desktop R environments, but temporary copies, metadata, raw transactions and concurrent users also consume memory.

10.3 Architecture refinement

- Retain matrices for dense, frequently aggregated numeric measures.
- Retain data frames for transaction-level and metadata-rich records.

- Avoid repeatedly converting large objects inside loops; convert once at the pipeline boundary.
- Use chunked ingestion or database-backed storage if raw transaction history becomes too large for memory.
- Benchmark with realistic sparsity and update frequency before selecting a final production storage engine.

TASK 11 – Defend the Final Architecture Under a Changed Requirement (4 Marks)

Question / requirement: Adapt the design for 10,000 stores, 100 products, three channels and frequent store-level stock/price updates; identify retained, modified, replaced or redesigned components; demonstrate an integrated new component; defend the design; and state trade-offs and future improvements.

11.1 Revised architecture decisions

Component	Decision	Reason
Store/product master data frames	Retain and strengthen keys	They support metadata, joins and validation.
Character IDs and logical flags	Retain	They preserve semantic meaning and avoid numeric coercion.
Region/category/channel factors	Modify with controlled level dictionaries	The expanded catalogue requires centrally managed allowed values.
Store × product matrix	Retain for selected dense snapshots	Efficient for numeric summaries, but not ideal for frequent individual updates.
Store × product × channel array	Extend	Represents the three required sales channels explicitly.
S3 Store class	Retain and add update methods	Centralizes business rules for stock and price changes.
Raw transaction data	Use long-form data frame or database table	A transaction table is naturally sparse and append-oriented.

11.2 Modified three-dimensional component

```
stores_10000 <- paste0("S", sprintf("%05d", 1:10000))
products_100 <- paste0("P", sprintf("%03d", 1:100))
channels <- c("in_store", "online", "mobile_app")

# A compact demonstration array; use zero initialization for a new snapshot.
channel_array <- array(
  0,
  dim = c(length(stores_10000), length(products_100), length(channels)),
  dimnames = list(
```

```
    store = stores_10000,
    product = products_100,
    channel = channels
  )
)

# Update one known cell using business identifiers
channel_array["S00001", "P001", "online"] <- 7
sum(channel_array["S00001", , ])
```

11.3 Explicit update component

```
update_inventory <- function(inventory, store_id, product_id,
                             new_stock, new_price = NULL) {
  required <- c("store_id", "product_id", "stock_qty", "unit_price")
  stopifnot(all(required %in% names(inventory)))
  if (length(new_stock) != 1 || is.na(new_stock) ||
      new_stock < 0 || new_stock != floor(new_stock))
    stop("new_stock must be one non-negative whole number")

  idx <- which(inventory$store_id == store_id &
               inventory$product_id == product_id)
  if (length(idx) != 1) stop("Expected exactly one inventory record")

  inventory$stock_qty[idx] <- as.integer(new_stock)
  if (!is.null(new_price)) {
    if (length(new_price) != 1 || is.na(new_price) || new_price < 0)
    stop("new_price must be one non-negative number")
    inventory$unit_price[idx] <- as.numeric(new_price)
  }
  inventory
}
```

This update function uses the store and product keys rather than row numbers. It checks that exactly one record matches, validates the new values, and returns the updated data frame. In a multi-user production platform, database transactions and concurrency controls would be required to prevent conflicting simultaneous updates.

11.4 Defence and remaining trade-offs

The revised architecture satisfies the changed requirement by separating the transaction/update layer from the analytical snapshot layer. The data frame or database table is the authoritative record for frequent updates; matrices and arrays are derived snapshots for fast summaries. Factors and validation functions protect category consistency. Named dimensions preserve the interpretation of the analytical cube.

Trade-off / limitation	Future improvement
Dense arrays allocate cells even when many store-product-channel combinations have no sales.	Use sparse matrices or store only observed transactions in a database.
R objects are generally processed in memory and may be copied during modification.	Use data.table, database queries, chunked processing or distributed processing when required.
A local R script does not coordinate concurrent updates from multiple users.	Use a transactional database with keys, constraints and audit logs.
Factors require controlled level updates when categories change.	Maintain a central category dictionary and validation service.

TASK 12 – Document, Package and Present the Final Framework (4 Marks)

Question / requirement: Prepare a data dictionary; create a reproducible script with seed, comments and session information; create a summary-report function; and provide a management recommendation and at least two lessons learned.

12.1 Data dictionary

Object	Class / typeof	Dimensions or length	Purpose	Validation rule
stores	data.frame / list	3 rows in example	Store master data	Unique non-missing character store_id; non-negative footfall
products	data.frame / list	3 rows in example	Product catalogue	Unique ID; non-negative numeric price and integer stock
store_record	list / list	Nested elements	One heterogeneous store record	Required identity and valid nested inventory
store_analysis$region	factor / integer codes	One value per store	Controlled region category	Only approved region levels
units_matrix	matrix / double	3 × 3 example	Store-product unit totals	Non-negative values; named row/column IDs
sales_array	array / double	3 × 3 × 2 example	Store-product-month totals	Correct dimension names and non-negative values
sales_ready	data.frame / list	One row per accepted record	Analysis-ready transaction data	Valid IDs, channel, units and price
central	RetailStore / list	Four fields	Validated store object	Constructor and update method enforce rules
benchmark_results	data.frame / list	3 rows in example	Scalability measurements	Positive dimensions; timings from actual run

Functions such as class(), typeof(), dim(), length() and str() can be used to inspect the objects. Example: class(units_matrix), typeof(units_matrix), dim(units_matrix), length(units_matrix), and str(sales_array).

12.2 Summary-report function

```
create_summary_report <- function(stores, products, sales, test_status) {
  list(
    generated_at = Sys.time(),
    store_total = nrow(stores),
    open_store_total = sum(stores$open_status, na.rm = TRUE),
    product_total = nrow(products),
    product_category_summary = table(products$category),
    sales_record_total = nrow(sales),
    units_sold_total = sum(sales$units_sold, na.rm = TRUE),
    revenue_total = sum(sales$revenue, na.rm = TRUE),
    validation_test_status = test_status,
    session = R.version.string
  )
}
```

12.3 Reproducible packaging instructions

6. Save the complete code in one file named smart_retail_framework.R.
7. Keep raw input files in a clearly named data folder and generated reports in an output folder.
8. Start the script with set.seed(2026), the working-directory setup and package checks.
9. Run the script from a fresh R session using source('smart_retail_framework.R').
10. Record sessionInfo(), benchmark_results and validation outcomes with the final submission.
11. Keep rejected raw rows and error messages for traceability; do not silently discard them.

12.4 Recommendation to retail management

Use character vectors for store and product identifiers, names and raw labels; numeric/integer vectors for prices, quantities and footfall; and logical vectors for status flags. Use data frames as the principal master-data and transaction layer because they combine different field types and support joins and filtering. Use factors for controlled categories such as region, product category and sales channel. Use lists for nested records and configuration. Use matrices for dense store-by-product numerical summaries and arrays for store-by-product-by-channel or month summaries. Use S3 classes and validated functions to centralize stock and revenue operations.

12.5 Lessons learned

- R's automatic coercion is convenient for simple vectors but dangerous when mixed business fields are combined. Separate columns in a data frame or elements in a list preserve each field's meaning.

- Matrices and arrays are efficient numerical structures, but they must carry meaningful dimension names and be converted to data frames when metadata-rich reporting is needed.
- Validation should occur before factor conversion and numerical analysis; otherwise invalid values may become NA or contaminate summaries.
- A scalable design is not only about memory. Frequent updates, concurrent users, auditability and maintainability influence whether an R object or a database should be authoritative.

CONSOLIDATED EXECUTABLE R SCRIPT

The following is a compact execution order for the principal components. The task sections above contain the full explanations and individual modules. Place the code blocks into one script in the order shown, keeping function definitions before their first use.

```
# Smart Retail R Framework - reproducible setup
set.seed(2026)
options(stringsAsFactors = FALSE)
R.version.string
sessionInfo()

# Create master data
stores <- data.frame(
  store_id = c("S001", "S002", "S003"),
  store_name = c("Central", "North Mall", "Airport"),
  region = c("South", "North", "South"),
  daily_footfall = c(1250L, 980L, 1500L),
  open_status = c(TRUE, TRUE, FALSE)
)
products <- data.frame(
  product_id = c("P001", "P002", "P003"),
  product_name = c("Rice", "Tea", "Soap"),
  category = factor(c("Grocery", "Beverage", "Personal Care")),
  unit_price = c(65.5, 120, 32),
  stock_qty = c(250L, 80L, 0L),
  promotion_status = c(FALSE, TRUE, FALSE)
)

# Matrix and array analytical structures
units_matrix <- matrix(
  c(12,8,5, 7,10,4, 15,6,9), nrow = 3, byrow = TRUE,
  dimnames = list(stores$store_id, products$product_id)
)
price_vector <- setNames(products$unit_price, products$product_id)
store_revenue <- units_matrix %*% price_vector

sales_array <- array(
  c(12,8,5, 7,10,4, 15,6,9, 11,7,3, 9,5,6, 8,12,4),
  dim = c(3,3,2),
```

```
  dimnames = list(stores$store_id, products$product_id, c("Jan","Feb"))
)

# Inspect structures
str(stores)
str(products)
dim(units_matrix)
dim(sales_array)
rowSums(units_matrix)
colSums(units_matrix)
apply(sales_array, 1, sum)

# Record reproducibility information
sessionInfo()
```

CONCLUSION

This assignment proposes an integrated R data-representation framework for a smart retail sales and inventory analytics system. The framework uses each R structure for the task it handles best: vectors for homogeneous fields, lists for heterogeneous records, factors for controlled categories, data frames for master and transaction data, matrices for two-dimensional numerical summaries, arrays for multi-dimensional analysis and classes/functions for reusable business rules.

The design also distinguishes R's permitted behaviour from application-appropriate behaviour. Although R can coerce mixed values into a common atomic type, the retail system should preserve identifiers, quantities, prices and status flags in separate typed fields. Named dimensions, validation, reproducible scripts and explicit conversion paths make the architecture easier to test and maintain. For frequent updates and concurrent use at national scale, an R analytical layer should be supported by a transactional database rather than treated as the sole system of record.